

Arduino Circuit Simulation

A Beginner's Guide to SimulIDE

August 23, 2026

Contents

1	Introduction	3
2	Software Required	3
2.1	Arduino IDE	3
2.2	SimulIDE	3
2.3	macOS Users: Rosetta	4
3	Recommended Software	4
3.1	Obsidian	4
3.2	GitHub	5
3.3	Using Git with Obsidian	5
4	How Arduino Simulation Works	5
4.1	Step 1 — Install the Arduino Board Package	5
4.2	Step 2 — Required Arduino Libraries	6
4.3	Other Useful Libraries for Future Projects	6
4.4	Examples of When You May Need These Libraries	7
4.5	Step 3 — Write the Arduino Program	8
4.6	Step 4 — Compile the Sketch	8
4.7	Step 5 — Save and Export the Compiled Binary	8
4.8	Step 6 — Load the HEX File into SimulIDE	9
4.9	Step 7 — Reload the Firmware	9
5	Task 1 — Digital Input and Output	10
5.1	Task 1.0 — Blink an LED	10
5.1.1	Problem Statement	10
5.1.2	How Does the Circuit Work?	11
5.1.3	What You Need to Do	11
5.1.4	Things to Think About	11
5.1.5	Expected Result	12
5.2	Task 1.1 — 7-Segment Counter Using a Push Button	12
5.2.1	Problem Statement	12
5.2.2	How Does the Circuit Work?	12
5.2.3	What You Need to Do	13

5.2.4	Things to Think About	13
5.2.5	Expected Result	13
5.3	Task 1.2 — LED Matrix Patterns	13
5.3.1	Problem Statement	13
5.3.2	How Does the Circuit Work?	14
5.3.3	What You Need to Do	14
5.3.4	Things to Think About	15
5.3.5	Expected Result	15
6	Task 2 — Temperature Measurement and Automatic Control	15
6.1	Task 2.0 — Digital Thermometer	15
6.1.1	Problem Statement	15
6.1.2	Learning About Communication Protocols	15
6.1.3	How Does the Circuit Work?	16
6.1.4	What You Need to Do	16
6.1.5	Things to Think About	17
6.1.6	Expected Result	17
6.2	Task 2.1 — Automatic Temperature Control	17
6.2.1	Problem Statement	17
6.2.2	How Does the Circuit Work?	17
6.2.3	What You Need to Do	18
6.2.4	Investigation — How Precise Is the Controller?	18
6.2.5	Investigation — How Can We Improve the Precision?	19
6.3	Task 2.2 — Smart Weather Forecaster	20
6.3.1	Problem Statement	20
6.3.2	How Does the Circuit Work?	20
6.3.3	What You Need to Do	21
6.3.4	Example LCD Output	21
6.3.5	Things to Think About	22
7	Common Problems	22
7.1	LED Does Not Turn ON	22
7.2	Button Does Not Work	22
7.3	7-Segment Displays the Wrong Number	22
7.4	LED Matrix Shows the Wrong Pattern	23
7.5	LCD Shows Nothing	23
8	Conclusion	23

1 Introduction

This guide explains how to design, program, and simulate simple Arduino circuits without physically building the circuit on a breadboard.

We will use:

- Arduino IDE — to write and compile Arduino programs.
- SimulIDE — to build and simulate electronic circuits.
- Obsidian — recommended for taking notes and organizing the project.
- GitHub — recommended for backing up and tracking changes to the project.
- LLM — obviously AI

The experiments are arranged from very simple to more advanced.

Beginner Note

You do not need to understand everything about Arduino before starting this guide. Each experiment introduces a few new concepts and builds on the previous experiment.

2 Software Required

2.1 Arduino IDE

Arduino IDE is used to write the Arduino program, compile it, and generate the program that will be loaded into the simulator.

Download Arduino IDE from:

<https://www.arduino.cc/en/software>

After installing Arduino IDE, open it once and make sure that it starts correctly.

2.2 SimulIDE

SimulIDE is the circuit simulator that we will use to simulate the Arduino circuits.

Download SimulIDE from:

<https://simulide.com/>

You can also use Fritzing:

<https://gist.github.com/RyanLua/fc2457d87641bb39754278b01a647526>

2.3 macOS Users: Rosetta

Some versions of SimulIDE are distributed for Intel-based macOS systems.

If you are using an Apple Silicon Mac, such as a Mac with an M-series processor, macOS may need Rosetta to run Intel applications.

If macOS asks you to install Rosetta, allow the installation.

You can also install it from Terminal using:

```
1 softwareupdate --install-rosetta
```

After installing Rosetta, try opening SimulIDE again.

Important

Rosetta is an emulation and translation layer that allows Intel-based applications to run on Apple Silicon Macs.

If SimulIDE does not start normally on an Apple Silicon Mac, check whether you are using the appropriate version and whether Rosetta is installed.

3 Recommended Software

The following software is not strictly required, but it is highly recommended.

3.1 Obsidian

Obsidian is useful for maintaining laboratory notes, circuit diagrams, explanations, code, screenshots, and links between experiments. Completely optional but recommended.

Download Obsidian from:

<https://obsidian.md/>

A useful project structure is:

```
Arduino-Simulation/  
|  
+-- Task-1/  
|   +-- LED-Blink/  
|   +-- Seven-Segment/  
|   +-- LED-Matrix/  
|  
+-- Task-2/  
|   +-- Temperature-Control/  
|   +-- Weather-Station/  
|  
+-- Images/  
|  
+-- Notes/  
|  
+-- README.md
```

3.2 GitHub

GitHub can be used to keep a backup of the project and to keep track of changes.

For example, after completing an experiment you can commit the Arduino code, circuit files, screenshots, and notes.

3.3 Using Git with Obsidian

The following resources explain how to connect an Obsidian vault with GitHub.

1. YouTube tutorial:

<https://youtu.be/5YZz38U20ws?si=329AtgfQyo9s3RXz>

2. Written guide:

<https://linked-blog-starter.vercel.app/connect-obsidian-vault-with-github>

Beginner Note

Git is optional for completing the experiments.
However, learning Git early is useful because your Arduino programs, simulation files, notes, and reports can all be kept together.

4 How Arduino Simulation Works

Before starting the experiments, understand the basic workflow.

The Arduino IDE is used to write the program, compile it, and generate the firmware file that will be loaded into SimulIDE.

The overall process is:

**Write Code → Select Arduino Board → Compile → Export HEX File →
Load into SimulIDE**

4.1 Step 1 — Install the Arduino Board Package

Before compiling a program, Arduino IDE needs to know which Arduino board you are using.

Open:

Tools → Board → Boards Manager

Search for the required board package.

For an Arduino Uno, search for:

Arduino AVR Boards

Install the package if it is not already installed.

After installation, select:

Tools → Board → Arduino Uno

Beginner Note

The selected board tells Arduino IDE which microcontroller, architecture, memory layout, and compiler settings should be used when compiling your program.

4.2 Step 2 — Required Arduino Libraries

Some experiments require additional Arduino libraries. These can be installed directly from the Arduino IDE Library Manager.

Open:

Tools → Manage Libraries

Search for the required library and click **Install**.

The following libraries are required for the experiments in this guide:

Library	Used For
DHT sensor library by Adafruit	Reading temperature and humidity from DHT11/DHT22 sensors
Adafruit Unified Sensor	Dependency required by the Adafruit DHT sensor library

Table 1: Required external Arduino libraries

Important

Install the **DHT sensor library by Adafruit** together with its required dependency, **Adafruit Unified Sensor**.

If Arduino IDE automatically asks to install a required dependency, allow it to do so.

4.3 Other Useful Libraries for Future Projects

The experiments in this guide require only a small number of libraries. However, as Arduino projects become more advanced, additional libraries may be required.

The following libraries are useful for common sensors, displays, storage devices, and other hardware.

Library	Typical Use
OneWire	Communication with OneWire devices such as DS18B20 temperature sensors
RTC	Working with real-time clock (RTC) modules
SdFat	Reading and writing data to SD cards
Adafruit SSD1306	Controlling SSD1306 OLED displays
Adafruit BusIO	Communication support used by several Adafruit libraries
Adafruit NeoPixel	Controlling addressable RGB LEDs such as NeoPixels
Adafruit PCD8544 Nokia 5110 LCD library	Controlling Nokia 5110 / PCD8544 graphical LCD displays
Adafruit SPIFlash	Working with SPI flash memory
Adafruit TinyUSB	USB functionality on supported Arduino boards
OneWire	Communication with OneWire sensors and devices

Table 2: Useful Arduino libraries for future projects

Important

Do not install all of these libraries just because they are listed here. Install a library only when the component used in your project requires it. This keeps the Arduino environment clean and avoids unnecessary dependencies.

4.4 Examples of When You May Need These Libraries

As you progress to more advanced projects, you may encounter situations such as:

- **DS18B20 temperature sensor:** Use the `OneWire` library along with the appropriate sensor library.
- **OLED display:** The `Adafruit SSD1306` library can be used for compatible SSD1306 displays.
- **Nokia 5110 LCD:** Use the appropriate `Adafruit PCD8544` library.
- **SD card data logging:** Libraries such as `SdFat` can be used for reading and writing files on an SD card.
- **Real-time clock:** An RTC library can be used when working with RTC modules.
- **Addressable RGB LEDs:** The `Adafruit NeoPixel` library can be used to control compatible addressable LEDs.

Beginner Note

A good rule is:

Component → Check its documentation → Install the required library

Different components may require different libraries, and some components can be controlled without any external library at all.

4.5 Step 3 — Write the Arduino Program

Create a new Arduino sketch and write your program.

For example:

```
1 void setup() {  
2     pinMode(13, OUTPUT);  
3 }  
4  
5 void loop() {  
6     digitalWrite(13, HIGH);  
7     delay(1000);  
8  
9     digitalWrite(13, LOW);  
10    delay(1000);  
11 }
```

Save the sketch before compiling it.

4.6 Step 4 — Compile the Sketch

Click the:

Verify / Compile

button in Arduino IDE.

Arduino IDE will compile the program for the selected board.

If there are no errors, the sketch has been successfully compiled.

4.7 Step 5 — Save and Export the Compiled Binary

After successfully compiling the program, save the sketch using:

Ctrl + Shift + S

Save the sketch in a suitable folder.

Then select:

Sketch → Export Compiled Binary

Arduino IDE will generate the compiled firmware files in the sketch folder.

For an Arduino Uno, look for the file ending in:

`.hex`

For example:

`Blink.ino.hex`

The HEX file contains the machine code that will be executed by the simulated Arduino.

Beginner Note

You do not need to understand the contents of the HEX file.
Think of it as the compiled version of your Arduino program that can be loaded into the simulated microcontroller.

Keep the `.hex` file somewhere easy to find because it will be loaded into SimulIDE in the next step.

4.8 Step 6 — Load the HEX File into SimulIDE

Open SimulIDE and create your Arduino circuit.

Place the required Arduino board and electronic components.

Once the circuit is ready, load the compiled firmware into the simulated Arduino.

Select the Arduino in SimulIDE and use the firmware/program loading option to select the generated:

`.hex`

file.

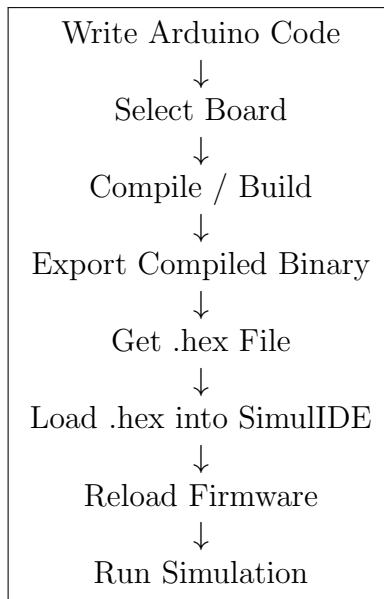
The simulated Arduino will now contain the same compiled program that was generated by Arduino IDE.

4.9 Step 7 — Reload the Firmware

After loading a new HEX file, reload or restart the firmware in SimulIDE before running the simulation.

This ensures that the simulated Arduino starts executing the newly loaded program from the beginning.

The complete workflow is therefore:



Important

Whenever you modify the Arduino program, you must compile it again, export the new HEX file, load the new HEX file into SimulIDE, and reload the firmware before testing the changes.

5 Task 1 — Digital Input and Output

In this task, you will learn how an Arduino communicates with electronic components using digital signals.

A digital signal has two basic states:

$$\text{LOW} = 0 \quad \text{HIGH} = 1$$

5.1 Task 1.0 — Blink an LED

5.1.1 Problem Statement

Your first task is to make an LED blink using an Arduino Uno.

Connect an LED to one of the Arduino's digital output pins.

Write an Arduino program that repeatedly:

1. Turns the LED ON.
2. Keeps it ON for a fixed amount of time.
3. Turns the LED OFF.
4. Keeps it OFF for a fixed amount of time.
5. Repeats the process continuously.

You should build and test the complete circuit in SimulIDE.

5.1.2 How Does the Circuit Work?

An LED produces light when current flows through it in the correct direction.

The Arduino's digital pin can produce two basic voltage states:

HIGH → approximately 5 V

LOW → approximately 0 V

When the Arduino pin is HIGH, current flows through the resistor and LED, causing the LED to glow.

When the pin is LOW, the voltage across the LED becomes approximately zero and the LED turns OFF.

The resistor is used to limit the current through the LED.

The basic circuit is:

Arduino Pin → Resistor → LED → GND

Beginner Note

Think of the Arduino digital pin as a small electronic switch.

HIGH = Switch ON

LOW = Switch OFF

The Arduino program simply controls this switch repeatedly.

5.1.3 What You Need to Do

1. Build the circuit in SimulIDE.
2. Choose a suitable Arduino digital pin.
3. Write the Arduino program.
4. Compile the program using Arduino IDE.
5. Load the program into the simulated Arduino.
6. Start the simulation.
7. Verify that the LED blinks continuously.

5.1.4 Things to Think About

Try changing the ON and OFF times.

For example:

- Make the LED blink very slowly.
- Make the LED blink very quickly.
- Make the ON time different from the OFF time.

Observe how changing the program changes the behaviour of the circuit.

5.1.5 Expected Result

The LED should repeatedly turn ON and OFF according to the timing specified in your program.

5.2 Task 1.1 — 7-Segment Counter Using a Push Button

5.2.1 Problem Statement

Design a digital counter using an Arduino Uno, a 7-segment display, and a push button. The counter should initially display:

0

Every time the push button is pressed, the number should increase by one:

$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow \dots \rightarrow 9$

After 9, the counter should return to 0:

$9 \rightarrow 0$

The counter should increase only when the button is pressed.

5.2.2 How Does the Circuit Work?

A 7-segment display is made from seven individual LED segments.

The segments are usually named:

a, b, c, d, e, f, g

By turning different segments ON and OFF, we can create different numbers.

For example, displaying 8 requires all seven segments to be ON.

Displaying 1 requires only two segments to be ON.

Therefore, the Arduino has to determine which segments should be ON for every number.

The basic idea is:

Arduino $\rightarrow$ 7-Segment Display

The Arduino uses several digital output pins to control the individual segments.

The push button provides the input:

Push Button $\rightarrow$ Arduino Digital Input

When the button is pressed, the Arduino detects the change in the input and increases the counter.

Beginner Note

The important idea is:

Button = Input
7-Segment = Output

The Arduino reads the input and changes the output.

5.2.3 What You Need to Do

1. Connect the seven segments to Arduino digital pins.
2. Connect the push button to a digital input.
3. Determine the segment combination required for digits 0–9.
4. Write a function that displays a particular digit.
5. Detect button presses.
6. Increase the counter after each valid button press.
7. Make the counter return to 0 after 9.

5.2.4 Things to Think About

Consider the following questions:

- How will you store the segment patterns for 0–9?
- How will you detect that the button was pressed?
- What happens if the button is held down?
- Why might a button appear to generate multiple presses?

The last question introduces the idea of:

Button Debouncing

5.2.5 Expected Result

The display should show one digit at a time.

Each valid button press should increase the displayed number by one, with the counter wrapping from 9 back to 0.

5.3 Task 1.2 — LED Matrix Patterns

5.3.1 Problem Statement

Interface an 8×8 LED matrix with an Arduino Uno and display different patterns.

Your first objective is to display a simple pattern such as:

T

After successfully displaying the first pattern, modify the program to display other patterns of your choice.

Possible patterns include:

- A
- B

- C
- Arrow
- Square
- Cross
- Smiley face

5.3.2 How Does the Circuit Work?

An 8×8 LED matrix contains:

$$8 \times 8 = 64$$

LED positions.

However, we do not normally need 64 independent Arduino pins.

The LEDs are arranged using rows and columns.

A particular LED can be selected by controlling its corresponding row and column.

The basic idea is:

Select Row → Set Column Pattern → Display

The Arduino rapidly repeats this process for every row.

Because this happens very quickly, our eyes perceive the complete pattern.

This technique is called:

Multiplexing

Beginner Note

Think of the LED matrix as an 8×8 pixel screen.

Each pixel can either be:

$$\text{ON} = 1 \quad \text{OFF} = 0$$

Therefore, an image can be represented using eight binary numbers, one for each row.

5.3.3 What You Need to Do

1. Connect the rows and columns of the LED matrix.
2. Determine how the matrix is oriented.
3. Create a binary pattern for a character or image.
4. Write the row-scanning code.
5. Display the pattern in SimulIDE.
6. Create at least three different patterns.

5.3.4 Things to Think About

Try to answer:

- Why do we need to scan the rows?
- What happens if the scanning is too slow?
- How can a binary number represent one row?
- How can you create your own character?

5.3.5 Expected Result

The selected pattern should appear as a stable image on the 8×8 LED matrix.

6 Task 2 — Temperature Measurement and Automatic Control

Now we move from digital input/output to an analog sensor.

The Arduino will measure temperature and make decisions based on the measured value.

6.1 Task 2.0 — Digital Thermometer

6.1.1 Problem Statement

Build a digital thermometer using an Arduino Uno, an LM35, DHT11 is recommended, temperature sensor, and a 16×2 LCD.

The Arduino should:

1. Read the analog output of the temperature sensor.
2. Convert the ADC value into temperature.
3. Display the temperature on the LCD.

For example, the LCD could display:

Temperature:

25.00 C

6.1.2 Learning About Communication Protocols

As you progress to more advanced Arduino and embedded-system projects, you will encounter different communication protocols used to transfer data between microcontrollers, sensors, displays, and other electronic devices.

Some common protocols include:

- UART: <https://youtu.be/sTHckUyxwp8?si=VORAq31n14zCPdpu>
- SPI: <https://youtu.be/0nVNwozXsIc?si=DqyBXqoA1RSWLB4y>

- I²C (I2C): <https://youtu.be/CAvawEcxoPU?si=MHu0B3mlRhXbLUfd>

Important

Before working with communication protocols, it is recommended to watch introductory videos to understand how these protocols work and how they differ from one another.

Rohde & Schwarz has useful videos explaining UART and other electronic communication concepts.

6.1.3 How Does the Circuit Work?

The LM35 is an analog temperature sensor.

It produces a voltage that changes with temperature.

Approximately:

$$10\text{ mV}/^{\circ}\text{C}$$

For example:

$$25^{\circ}\text{C} \rightarrow 250\text{ mV}$$

The Arduino cannot directly understand a voltage as a number.

It uses an:

ADC (Analog-to-Digital Converter)

to convert the sensor voltage into a digital value.

For the Arduino Uno's 10-bit ADC:

$$0 \rightarrow 1023$$

The Arduino then converts this value into voltage and finally into temperature.

The overall process is:

$$\text{Temperature} \rightarrow \text{LM35 Voltage} \rightarrow \text{ADC} \rightarrow \text{Arduino} \rightarrow \text{LCD}$$

6.1.4 What You Need to Do

1. Connect the LM35 output to an Arduino analog input.
2. Connect the LCD.
3. Read the ADC value using `analogRead()`.
4. Convert the ADC value into voltage.
5. Convert voltage into temperature.
6. Display the result on the LCD.

6.1.5 Things to Think About

Calculate the theoretical ADC resolution of the Arduino Uno.

Then determine how many ADC counts correspond approximately to 1°C when using an LM35.

6.1.6 Expected Result

The LCD should display the temperature measured by the LM35.

6.2 Task 2.1 — Automatic Temperature Control

6.2.1 Problem Statement

Extend the digital thermometer into an automatic temperature-control system.

The system should control a heater and cooling devices according to the measured temperature.

Your system should:

- Turn ON an indicator LED when the temperature becomes too high.
- Turn ON a DC fan when the temperature becomes too high.
- Control an AC fan through a relay in the simulation.
- Turn ON a heater when the temperature becomes too low.
- Display the current temperature on the LCD.

For example, you may define:

$$T < T_{low} \Rightarrow \text{Heater ON}$$

$$T_{low} \leq T \leq T_{high} \Rightarrow \text{No heating/cooling}$$

$$T > T_{high} \Rightarrow \text{Cooling ON}$$

6.2.2 How Does the Circuit Work?

The temperature sensor provides the input.

The Arduino continuously measures the temperature and compares it with predefined limits.

The Arduino then decides which output should be activated.

The overall system is:

$$\text{Temperature Sensor} \rightarrow \text{Arduino} \rightarrow \text{Decision} \rightarrow \text{Actuator}$$

The actuators can include:

- LED

- Heater
- DC fan (use L293/L293D because in real life you can't run a motor on Arduino alone because of current limitation)
- or Relay-controlled AC fan

For example:

$$T > 35^{\circ}\text{C} \Rightarrow \text{Cooling Fan ON}$$

and:

$$T < 25^{\circ}\text{C} \Rightarrow \text{Heater ON}$$

Important

The Arduino GPIO pin should not directly drive a motor, heater, or AC appliance. In a real circuit, suitable driver circuitry such as a transistor, MOSFET, motor driver, or relay would be required.

The AC fan in this task is intended for simulation and control logic practice.

6.2.3 What You Need to Do

1. Build the temperature-sensing circuit.
2. Add the LCD.
3. Add a heater indicator.
4. Add a DC fan.
5. Add the required driver circuit.
6. Define suitable temperature thresholds.
7. Write the control program.
8. Test low, normal, and high temperatures.

6.2.4 Investigation — How Precise Is the Controller?

The controller uses on-off control.

This means that the heater is either:

ON or OFF

It does not continuously control the amount of heat.

Record the temperature while the system is operating.

Find:

$$T_{\max}$$

and:

$$T_{\min}$$

Then estimate the temperature variation.

For example, if:

$$T_{\max} = 26.2^{\circ}\text{C}$$

and:

$$T_{\min} = 24.8^{\circ}\text{C}$$

then:

$$\Delta T = 26.2 - 24.8 = 1.4^{\circ}\text{C}$$

and the approximate variation around the midpoint is:

$$\pm 0.7^{\circ}\text{C}$$

Important

Do not assume that ADC resolution is the same as temperature-control precision. The actual control performance is affected by the sensor, noise, thermal inertia, heater power, sampling rate, hysteresis, and the physical arrangement of the system.

6.2.5 Investigation — How Can We Improve the Precision?

Research and compare different approaches for improving temperature control.

Consider:

1. Smaller hysteresis
2. Sensor calibration
3. Higher-resolution ADC
4. Better voltage reference
5. Better thermal design
6. Proportional control
7. PID control

For your survey, compare:

- How the method works
- Advantages
- Disadvantages
- Expected precision
- Implementation complexity

6.3 Task 2.2 — Smart Weather Forecaster

6.3.1 Problem Statement

Build a smart weather-monitoring system using an Arduino Uno, temperature and humidity sensor, LCD, LEDs, fan, and buzzer.

The system should continuously monitor the environment and classify the weather into different conditions.

The system should detect:

- Normal weather
- Hot weather
- Cold weather
- Heat wave
- Extreme temperature
- High humidity / possible rain

The LCD should display the detected weather condition.

6.3.2 How Does the Circuit Work?

For this experiment, use a DHT22 sensor.

Unlike the LM35 used in the previous task, the DHT22 provides temperature and humidity data digitally.

The Arduino reads:

Temperature	Humidity
-------------	----------

and uses these values to determine the weather condition.

The overall system is:

DHT22 → Arduino → Weather Analysis → LCD + LEDs + Fan + Buzzer

For example, the system can use rules such as:

$$T > 35^{\circ}\text{C} \Rightarrow \text{Hot Weather}$$

$$T < 15^{\circ}\text{C} \Rightarrow \text{Cold Weather}$$

$$T > 45^{\circ}\text{C} \Rightarrow \text{Extreme Temperature}$$

$$T > 40^{\circ}\text{C} \text{ and } RH < 30\% \Rightarrow \text{Heat Wave}$$

$$RH > 80\% \Rightarrow \text{Rain Possibility}$$

Beginner Note

This experiment combines several ideas learned earlier.
You already know how to:

- read a sensor,
- control LEDs,
- control a motor,
- display information on an LCD,
- use `if--else` conditions.

Now you will combine them into one complete system.

6.3.3 What You Need to Do

1. Connect the DHT22 sensor.
2. Connect the LCD.
3. Add indicator LEDs.
4. Add a buzzer.
5. Add a cooling fan.
6. Read temperature and humidity.
7. Define the weather conditions.
8. Display the detected condition on the LCD.
9. Activate the appropriate indicators.

6.3.4 Example LCD Output

For normal weather:

NORMAL
WEATHER

For hot weather:

HOT WEATHER
TEMP HIGH

For cold weather:

COLD WEATHER
TEMP LOW

For extreme temperature:

EXTREME TEMP
ALERT

6.3.5 Things to Think About

Consider:

- What should happen if both temperature and humidity are high?
- Which condition should have the highest priority?
- Why should the extreme-temperature condition be checked before the normal hot-temperature condition?
- How can you add another weather condition?

7 Common Problems

7.1 LED Does Not Turn ON

Check:

- LED polarity
- Resistor connection
- Arduino pin number
- Ground connection
- Whether the program was loaded

7.2 Button Does Not Work

Check:

- Button wiring
- Input pin
- INPUT_PULLUP
- Ground connection

7.3 7-Segment Displays the Wrong Number

Check:

- Segment order
- Common-anode/common-cathode type
- Segment polarity
- Segment pattern

7.4 LED Matrix Shows the Wrong Pattern

Check:

- Row order
- Column order
- Matrix polarity
- Binary pattern
- Multiplexing code

7.5 LCD Shows Nothing

Check:

- Power
- Ground
- Contrast
- LCD pin mapping
- `lcd.begin()`

8 Conclusion

The experiments in this guide gradually introduce Arduino programming and electronic circuit simulation.

The progression is:

Digital Output → Digital Input → Displays → Multiplexing

followed by:

Analog Input → ADC → Sensors → Automatic Control

and finally:

Multiple Sensors → Decision Making → Complete Embedded System

The goal is not simply to copy a circuit or program.

The goal is to understand the relationship between:

Input → Processing → Output

Once this basic idea is understood, more complicated Arduino projects become much easier to design.